

Swarmbly AI

An artificial intelligence built by everyone

Public explainer Sebastián A. Espinoza-Ulloa, Ph.D. · 14 August 2026 · updated 29 September 2026

The essentials in one paragraph

Today, using advanced artificial intelligence means depending on a handful of companies that own enormous data centres. Swarmbly AI proposes something different: that AI should run by spreading the work across the computers we already have at home and in the office, most of which sit switched on all day doing almost nothing. The technical idea that makes this possible comes from an unexpected place — from the way biologists reconstruct a whole genome out of thousands of small fragments of DNA.

The name, in one paragraph

Swarmbly is two words: *swarm* and *assembly*. The swarm is the easy half — getting many computers to work at once is a problem that was solved decades ago. The assembly is the hard half, and it is where this project can fail: a swarm does not produce an answer, it produces *fragments*, and turning many independent fragments into one text a person can read without noticing the joins is the whole difficulty. That second half is borrowed from genetics, where reading a chromosome is impossible directly, so it is shattered into millions of pieces, each read separately, and the original is rebuilt from the overlaps. The name says what the system does — it swarms, and it assembles — and deliberately says nothing about where the idea came from, because that belongs in the technical paper where it can be stated precisely.

1. The problem is not knowledge. It is money.

There is a widespread confusion about why artificial intelligence is concentrated in so few hands. Many people assume the big companies are keeping a secret: a formula, an algorithm nobody else knows.

They are not. A great many AI models are published openly. Anyone can download them. The methods are explained in scientific papers that are there for everyone to read. The knowledge circulates.

What does not circulate is the money to **run** them. Putting a large model to work answering questions for millions of people demands graphics accelerators that cost tens of thousands of dollars each, entire buildings to house them, electricity contracts the size of a small city's, and internal networks running at speeds that do not exist outside those facilities.

That is where the real concentration lies. A technology whose **knowledge** is public ends up controlled by whoever can pay for the **hardware**.

The figures give the scale of it. Data centres consumed roughly 415 terawatt-hours of electricity in 2024, about 1.5% of all the electricity in the world, and projections put them near 945 by 2030. The large American centres are also fed by electricity grids dirtier than the national average: around 545 grams of CO₂ per kilowatt-hour against 370.

It is the portrait of an industry whose path to growth runs through building. And building is only within reach of those who can finance it.

2. And yet the hardware already exists

Here is the fact that gives the whole project its point.

There is a community of people who for decades have lent the spare time of their computers to scientific projects — searching for radio signals, folding proteins, running climate models. That system, called BOINC, brings together today around **700,000 active devices, four million processor cores and 560,000 graphics cards**, with an average power of 93 petaflops.

Now, the important part: that figure comes from a community that **has been shrinking** for twenty years, from close to a million volunteers down to some two hundred thousand. Which means it is not a ceiling. It is a floor, measured in a niche in retreat.

And underneath those organised volunteers there are hundreds of millions of personal computers, laptops and office machines that wake their graphics card up for a while each day and leave it idle the rest of the time. That hardware has already been manufactured. It already draws electricity just by being switched on. It is already paid for.

The world's spare computing capacity is not a hypothesis. It exists. What is missing is a way to use it.

3. Why nobody has managed it yet

There have been serious attempts, and the best known of them — a project called Petals — deserves credit: it showed that an AI spread across volunteers over the internet is possible. Without it, this proposal would have nothing to stand on.

But all those attempts share one design decision that limits them: **they split the model**.

Imagine the AI model is an enormous brain. These systems slice it up and hand one slice to each participant. To produce **a single word** of the answer, the thought has to travel across the internet from the first slice to the second, from the second to the third, and so on to the end. And then it starts over for the next word.

The problem is distance. Inside a data centre, graphics cards talk to each other at around 900 gigabytes per second. The upload connection of a typical home is about 60 megabits per second. The difference is roughly **120,000 times**. In response time the gap is four to five orders of magnitude: microseconds inside the data centre, tens or hundreds of milliseconds between cities.

Crossing that chasm **once for every word** is unsustainable. And the measurements confirm it: Petals loses around 31% of its performance simply by moving from a data-centre connection to a home connection.

It is not that they did it badly. It is that they are answering the wrong question well.

4. The Swarmbly idea: split the problem, not the brain

Swarmbly changes the question. Instead of *"how do we make one giant brain run across many machines?"*, it asks:

How do we put many small, complete brains to work on one big problem?

Each participant runs a **whole** model — small, but complete and self-contained, the size that fits comfortably on an ordinary computer. What gets divided up is not the brain: it is **the task**.

And because each participant receives a complete assignment and returns a complete answer, **information crosses the internet once per assignment**, not once per word. That change is what lifts the system out of the chasm described in the previous section. It is not an improvement on what came before: it is a different regime.

5. Where the idea comes from: how a genome is read

This is where the project borrows its central idea from biology, and it is worth telling properly, because it is the heart of everything.

A human genome has about three billion letters. No machine can read it straight through. What laboratories have done for decades is something that sounds absurd the first time you hear it: **they break the DNA into millions of small pieces at random**, read each little piece separately, and then reconstruct the complete sequence out of that heap of fragments.

It works for two reasons.

The first is **redundancy**. Each region is not read once but many times. Every point in the genome shows up in several different fragments, and that repetition is what makes it possible to correct errors: if nine reads say one thing and one says another, the odd one out was a reading error.

The second is **overlap**. The fragments run over each other at the edges, and that overlap is what tells you how they fit together, exactly as in a jigsaw puzzle.

Swarmbly applies that same logic to language.

In genomics	In Swarmbly
A genome that has to be read	A task that has to be solved
DNA fragments (<i>reads</i>)	Micro-tasks sent to different computers
Overlap between fragments	Shared context between micro-tasks
Reconstruction of the genome	Assembly of the answer on your computer
Low-quality regions flagged	Unreliable parts of the answer, flagged

And there is a reason this analogy is not decoration: the author of the proposal holds a doctorate in population genomics and assembles genomes for a living. The idea does not come from a metaphor borrowed from outside. It comes from inside the trade.

6. How it works, step by step

Suppose you ask the system for a long report on some subject.

First, your computer decides whether splitting is worth it. A small model running locally sizes up the request. If you ask something simple and direct — "what's the temperature today?" — it does **not** split it: it answers on the spot. This matters more than it seems, and I explain why in the next section.

Second, if the task is divisible, it plans. It breaks the work into parts that stand on their own — introduction, background, analysis, conclusions — and works out which parts depend on others and which can be done at the same time.

Third, it writes a shared contract. Before sending anything out, it writes a short note that travels with *every* assignment: what the goal is, who it is for, in what tone, in what format, what each thing is called. This is what stops one part coming back written like an academic paper and the next like a blog post.

Fourth, it distributes. Each micro-task travels to a different volunteer computer. And when a part matters, **the same task is sent to several participants at once**, deliberately choosing machines that run **models from different families**.

Fifth, it verifies. The system checks that each participant really used the model it claimed and not a smaller, cheaper one. There are methods that make this check with very high reliability at almost no cost.

Sixth, it assembles. Your computer puts the pieces together. When there are several answers to the same task, it **aligns them against each other** — just as a biologist aligns DNA reads — and produces a consensus version.

And seventh, it tells you what it trusts. That is what comes next, and it is the most interesting thing about the project.

7. The confidence map: something centralised AI cannot give you

When an AI model answers you today, it hands you a text that sounds equally sure of itself from beginning to end. The parts it knows well and the parts it is making up have exactly the same tone.

In Swarmbly, because the same task was solved by several **different and independent** models, their answers can be compared against each other. And that comparison says something:

- Where several models with nothing in common **agree**, there is a convergence that means something.
- Where they **disagree**, there is a clear signal that this point deserves a second look from you.

The system then delivers, alongside the answer, **a map of the unreliable regions**. Just as a genome assembler does not hand over a uniformly confident sequence, but marks the regions where the reading was doubtful.

And here is the remarkable part: **a centralised provider cannot offer this**, however much money it has. With a single model there is nothing to compare against. Asking one model ten times measures its indecision, not the disagreement between independent observers.

The redundancy that decentralisation **needs** turns out to be exactly what produces this information. A cost of the architecture and a capability of it are the same mechanism seen from two sides.

With one honest warning: **several models agreeing does not guarantee they are right**. If they all learned from the same faulty material, they can be wrong together. That is why the design insists on using models from different families, and why the relationship between "they agree" and "it is correct" is framed as something **to be measured**, not something taken for granted. If measuring it shows the relationship does not hold, that will have to be said.

It has now been measured, four times, and the relationship did not hold. So it is being said: the claim that agreement predicts correctness is **withdrawn** — see section 11. What survives of this section is the mechanism and nothing more. The system can still tell you where independent models disagreed, and that remains something a single centralised model cannot do; what it may not tell you is that the places they agreed are the places it got right. Read the rest of this section with that correction applied to it.

8. Why the system sometimes refuses to split

This point deserves a section of its own because it is counterintuitive, and because understanding it is understanding the real limits of the proposal.

Splitting does not always help. There are requests that **cannot** be divided without being destroyed.

Think of "what's the temperature today in Saskatoon?". Someone might think of chopping it up: send "what's the temperature?" to one computer, "today?" to another, "in Saskatoon?" to a third, and put the answers together.

It does not work, and the reason is precise. Whoever receives "what's the temperature?" will return a temperature of nowhere and of no particular moment. It is not a partially correct answer that can be completed later: it is an answer to a different question.

In genomic terms: a DNA fragment is a **contiguous, complete stretch** of the molecule. Chopping that question up is not fragmenting into pieces — it is taking a fragment and **deleting one letter in every three**. No amount of redundancy recovers that, because it is not sampling: it is information destroyed before sampling begins.

That is why Swarmbly splits at two distinct levels:

- **Macro level**, for large tasks that genuinely do have parts that stand on their own: a report with sections, a batch of a thousand documents.
- **Micro level**, where **the complete task** is sent to several participants and their answers are compared.

An atomic question jumps straight to the micro level. And a system that chops up atomic questions is, according to the protocol specification, **malfunctioning**.

9. Three circles: where your information actually goes

The previous sections describe *what* Swarmbly does with a request. This one describes *where* it goes, because those are different questions and confusing them is how privacy promises get broken.

Before anything is split, a small program on **your own machine** reads the request and decides which of three circles it belongs to. That program never asks the network for help — a privacy check that phones home to ask whether your text is private has already handed over your text.

The outer circle: the open network. Ordinary requests — a blog post, a summary of a public article, code you were going to publish anyway — go to the volunteer mesh described so far. Many unknown machines, full redundancy, full verification.

The middle circle: a trusted swarm. A company, a hospital, a university or a research group can run its own closed mesh. Only machines on an explicit list can join, every connection proves the identity of *both* ends, and the whole thing typically lives on the organisation's own network. It is the same software and the same protocol as the outer circle — the only thing that changes is who is allowed in.

This is the difference between "cannot be used with patient records" and "can be used with patient records, on the hospital's own machines". Data-protection law is written around organisations you can name and hold responsible; an anonymous volunteer can never be one, and a machine on the hospital's own list always can.

The inner circle: only your machine. For the most sensitive material, nothing leaves at all. Your computer does the whole job by itself. It is less capable, and that is the honest price of the guarantee — it is also the only case where the project makes an absolute promise about secrecy, and it can make it only because there is no network involved.

You can pick a circle by hand, and your choice always wins. If you have not picked, the local check errs on the side of caution: when it sees something that looks like a medical record, a password, a bank account or a legal case, it moves the request inwards on its own. It is deliberately over-cautious, because sending a harmless request to a closed circle costs a little speed while the opposite mistake costs the thing the circle exists to protect.

One honest catch, worth stating plainly. In a closed circle you can ask each task of a single machine instead of several, because the machines are known and there is no reason to expect any of them to lie. But comparing several answers is exactly how the confidence map of section 7 is built. Ask one machine and you save time and lose the map. The system will not do this quietly: when it happens, the answer comes back explicitly saying that no confidence map was produced. Choosing between speed and knowing what to double-check is a real decision, and it belongs to the person running the network, not to the software.

And a closed circle does not remove trust — it moves it. Whoever decides who is on the list holds that power, and a compromised machine *inside* the circle is more dangerous than an unknown machine outside it, precisely because fewer answers are being compared. The project says this in its own specification rather than leaving it to be discovered.

10. What people get out of it

Using AI without being able to buy the hardware. For anyone with a powerful graphics card, there are techniques that speed models up locally. For anyone without one, those techniques are of no use at all. The relevant comparison is not "faster or slower", it is "**I can do this or I cannot**". Saying the bus is slower than the car is only an argument if you have a car.

Capacity that grows with people, not with capital. Every person who joins brings capacity with them. It is a growth curve no company can match, because theirs is bounded by what it

can build and finance.

Tasks that are unworkable today because of their size. Processing ten thousand documents, going through a whole archive, analysing an entire corpus. Jobs that are billed dearly precisely because of their volume, and that here are shared out among many.

Knowing what to trust. The confidence map from section 7 — a mechanism that works, but one whose usefulness the measurements have now **withdrawn** rather than confirmed (section 11). It is named here rather than quietly dropped, because this document advertised it before it was tested.

A system you can audit instead of believe. The protocol is public, the software is public, and the system reports for itself how much quality it lost by splitting. Not as a courtesy: as a design requirement.

Making use of what has already been manufactured. Most of AI's environmental footprint is not in the electricity it burns while running, but in **manufacturing the hardware**: mining, manufacture, transport. Using machines that already exist avoids manufacturing new ones.

On this last point an honest qualification is in order. By making AI cheaper and easier, people will probably use it a great deal more — this is a known phenomenon: making a resource cheaper tends to raise its total consumption. Swarmbly's answer is not that this demand will disappear, but that **it can be absorbed by hardware that already exists**, without building new data centres. As long as there is idle capacity available, the additional environmental cost of serving it approaches zero.

11. What the first measurements showed

Update, September 2026. Since this section was written, a much larger round of tests has been run: 473 runs, with five different families of small models. Two things came out of it. The first is a cost: when both ways of working are given the same room to write, splitting a task still loses some quality — about 11 % on average — although the software that checks the results refuses to call that final, because the two ways of working also write different amounts, and that alone moves the score. The second is the result the design was built around. On a task where a small model has to read many records out of a long document and answer questions about all of them, letting the model only read, and letting ordinary code do the counting, gave far better answers than asking the same model to do everything at once. And when the document was long enough, reading it in pieces was the only way the small model kept up: in one go, it lost track. That is exactly the situation Swarmbly is meant for. It was measured on a small set of documents, with one model family, and the full account is in the whitepaper, version 2. What follows is the earlier round, kept as it was written.

The project has now been run against real language models — small models on one laptop. Two predictions were put to the test. **Neither of them passed**, and the reasons are different in each case. Both are below.

The central prediction was tested properly, and the project's own threshold was not met. The whole design rests on a claim: the more shared context each participant receives, the less quality is lost when the pieces are joined. Before any data existed, the project wrote down what would count as good enough — the loss had to be **below 5 %**, and, importantly, it

had to be below 5 % *even allowing for the uncertainty in the measurement*, not merely on average.

The best measurement now available, on a fresh set of 16 tasks that were held back so no threshold could be tuned to them: splitting a task in two costs about **2.30 %** of quality on average. That looks like a pass. It is not. Because 16 tasks is a small number, the honest statement of that result is a range rather than a single figure, and the range runs from **-2.05 % to +7.49 %** — its top end is above 5 %. The threshold was deliberately written against the top end of the range, precisely so that a flattering average could not carry the claim on its own. So the verdict is **not met** — short by 2.49 points on the figure that counts — and the project is reporting that rather than moving the goalposts it set for itself.

What is genuinely interesting is *underneath* the average, and it is a new finding. **The typical task lost nothing at all**: the middle task of the sixteen came out at exactly **0.00 %**, and **11 of the 16 tasks were at zero or better** — six of them actually improved, five came out exactly even. The average is not merely dragged up by the expensive tasks; it is *made* by two of them. Two tasks out of sixteen cost **+28.50 %** and **+23.08 %**, which between them is **140 %** of the whole average. Take those two out and the remaining fourteen average **-1.06 %** — splitting in two is very slightly *better* than not splitting at all. So the honest sentence is not "splitting costs 2.3 %". It is: **on 11 of the 16 tasks, splitting the work in two was free; on two of them it was expensive.**

There is a clue about those two, and it is worth naming. They are also the only two tasks where splitting into **two** pieces was worse than splitting into **eight** (+28.50 % against +13.57 %, and +23.08 % against +14.00 %). If the problem were simply that fragmentation breaks coherence, more fragments would hurt more, not less. So the likelier explanation is that the split itself was made in a bad place on those two tasks — a cutting problem rather than a cost of cutting. That is a clue, not a tested claim, and the project has not tested it.

None of this rescues the threshold. It does change what the result is *about*: for a system that decides task by task whether to split, a cost that is free most of the time and expensive occasionally is more useful than a cost that is small and uniform, because you can learn to route around the expensive cases. But the bar the project set for itself has not been cleared, and the honest report says so. A sanity check was also built into the run — a deliberately over-split arrangement that was *required* to fail, so that a broken instrument could not quietly pass everything. It failed as required, at **16.23 %**, with 15 of the 16 tasks coming out worse than not splitting at all.

One more limit worth stating plainly: with tasks this different from one another, sixteen of them simply cannot produce a range narrow enough to sit under a 5 % threshold, no matter what the true cost is. Testing the threshold again requires a wider set of tasks first. That is a precondition, not a way of getting a nicer answer.

An earlier version of this document reported something much better, and it was wrong. It said the central prediction held, and that the quality lost fell steadily from about 24 % down to about 14 % as the shared context rose. That result is **withdrawn**. The fault was not in the architecture but in the instrument measuring it — it was measuring the wrong thing, in two separate ways.

The first is the more embarrassing. The dial that was supposedly being turned — how much shared context each participant gets — was never actually turning. Every packet of work has a floor: the task itself and its instructions have to be in there, and they cost something before a single word of shared context is added. Set the budget below that floor and there is no room for context at all, so every setting produces exactly the same packet. On that first run, 13 of

96 measurements were above the floor; the two settings that anchored the whole downward trend had **none**. So the elegant descending curve was really saying "packets containing nothing do worse than packets containing something" — true, unsurprising, and not the claim being tested.

The second: the quality score itself was unfair to the split-up version. Handed the *very same* piece of text, it scored it 0.9375 when told it was written in one piece and 0.5000 when told it came in eight — an apparent penalty of **+46.7 %** on text that had not changed at all. An unknown share of that old 24 %-to-14 % curve was simply this.

Both faults are now fixed, both have tests that will catch them if they ever return, and the software now **refuses to publish** a measurement taken below the floor rather than printing it with a footnote.

The second prediction, the confidence map, is withdrawn outright. Section 7 of this document describes it as the thing centralised AI cannot give you: the system compares what several independent models said and flags where they disagree. The mechanism works exactly as designed — it really does find the disagreements. The claim that failed is the useful one: that where the models agree, the answer tends to be *right*.

The first measurement found no relationship at all between how much the models agreed and how good the answer was. At that point the fair verdict was "unproven, not disproven", because the test was weak — the automated marker accepted 93 % of everything it saw, leaving almost nothing to correlate against. So the experiment was rebuilt properly, marking answers against a known-correct answer key, and run three more times. The three runs disagreed with each other. Measured as the odds of an answer being correct, agreement multiplied those odds by **3.47** in the first run — a real, useful effect if true. In the second it multiplied them by **0.26**, meaning agreement made a correct answer *less* likely. In the third, **1.24**, which is indistinguishable from no effect at all. Above the line, below it, and sitting on it, on the same question.

That is not a weak signal. It is no signal, measured three times. So the confidence map is **withdrawn**, not merely unproven, and it has been dropped from the current round of testing. It was, until these measurements, the feature this project was proudest of — and the honest thing to do with a favourite idea that fails is to say so, in the same document that praised it. The mechanism is still there and still reports where the models disagreed; what it may no longer do is tell you that agreement means correctness.

12. What this does not promise, and what could still go wrong

A project that only tells you its virtues does not deserve trust. Both halves of the honest account are gathered here, in one place, rather than spread across the document.

Four things Swarmbly does not claim. It is **not faster than a commercial AI** for someone who already owns the equipment to use one. It **does not have infinite memory**: the limit on how much text can be handled at once does not vanish, it moves from a company's server to your own computer. That is a much higher ceiling, and it rises when you improve your machine rather than when you change your payment plan — but it is a ceiling. It is **not a secret system**: splitting information into pieces makes it harder for anyone to put it back together, but it is not encryption and it would be dishonest to call it that; for genuinely sensitive tasks — health, legal matters, personal data — the design does not send them to

unknown computers, it keeps them inside a closed circle or on your own machine, as section 9 describes. And it **has not yet demonstrated its environmental benefit**: the argument is sound, but the project commits to measuring it against a public standard and publishing the result whatever it turns out to be.

Two risks worth naming. The technical one is that splitting a task up and putting it back together costs quality. How much it costs was an open question; the project built the instrument that measures it, fixed the failure threshold before there were any data — and the best measurement now available comes back on the **wrong** side of that threshold, at one scale, on 16 tasks (section 11). It is close, and 11 of the 16 tasks lose nothing at all, but close is not the standard the project wrote for itself. The human risk is the bigger one: all of this depends on there being people willing to lend their computers, and volunteer computing has been shrinking for twenty years. It is considerably more likely that the project fails for want of participants than through an engineering fault. Having the best protocol in the world is worth nothing if nobody connects.

That second risk is also the reason this document exists.

13. Why it matters

There is an asymmetry at the centre of all this. The knowledge to build artificial intelligence is public. The capital to run it is not. And that difference — not a secret, not a patent — is what concentrates control over a general-purpose technology.

Meanwhile, the hardware capable of running that intelligence is switched on and idle in hundreds of millions of homes and offices. Roughly 700,000 machines are already enrolled in volunteer computing today, from a community that has been shrinking for twenty years. That figure is a floor, not a ceiling.

Swarmby is a proposal for connecting those two facts. If it works, the result is not a cheaper way of buying what is already for sale. It is computing capacity that **grows with the number of people taking part** instead of growing with the money available to build — and no company can match that curve, because a company's capacity is bounded by what it can finance and a network's is bounded by how many people want it to exist.

That is worth saying plainly: this is an attempt to change who gets to run a general-purpose technology, not an attempt to sell a discount version of it. The first measurements say the idea is not obviously wrong, which is more than most proposals of this size can claim on their first day.

If it does not work, that will be known, because the project published in advance how to check it and under what conditions it would call it a failure.

It is worth trying even knowing it may fail. And it is worth trying in plain sight, where anyone can examine it.

A minimal glossary

Language model — The program that generates text. There are large ones (the commercial ones) and small ones (the ones that fit on an ordinary computer).

Node — A computer taking part in the network by lending capacity.

Micro-task — Each of the assignments a large request is divided into.

Orchestrator — The program that runs on *your* computer: it decides whether to split, distributes the work, and puts the pieces back together.

Shared contract — The short note that travels with every assignment so that the pieces fit together.

Assembly — Joining the returned pieces into a single, coherent answer. The term comes from genomics.

Alignment — Comparing several answers to the same task to see where they agree and where they do not.

Confidence map — The report that accompanies the answer, pointing out which parts are reliable and which are worth reviewing.

Trusted swarm — A closed mesh whose members are on an explicit list, kept by an identified operator, with every connection proving the identity of both ends. Same protocol as the open network, restricted membership.

Privacy circle (tier) — Which population of machines a request is allowed to reach: the open network, a trusted swarm, or your machine alone. It is a separate question from how sensitive the content is, and both are decided before anything is sent.

This document explains the project for a general audience. The full technical description, with its mathematical foundations, its protocol specification and its references, is in the accompanying technical paper.

Spanish version: DIVULGACION_ES.md